

FACULDADE DE TECNOLOGIA DO ESTADO DE SÃO PAULO
DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

ARTEFATOS DO PROJETO DE SOFTWARE

Criação de Camarão em Cativeiro com Monitoramento IoT para Gastronomia

Davi Mathais de Almeida
Leandro Augusto De Souza Muniz
Tiago de Lara Rodrigues

Índice

1	CANVAS: Modelo de Negócio e Estratégia	3
1.1	Análise do Modelo de Negócio	3
2	SWOT: Análise Estratégica	4
2.1	Recomendações Estratégicas	5
3	UX/UI: Design de Interface e Experiência do Usuário	6
3.1	Guia de Estilos	6
3.2	Tipografia	6
4	Website: Estrutura e Especificações Técnicas Web	6
4.1	Mapa do Site da Equipe	7
4.2	Conteúdo das Páginas	7
4.3	Especificações Técnicas do Site da Equipe	7
5	Diagramas UML: Modelagem Visual do Sistema	8
5.1	Diagrama de Casos de Uso	8
5.2	Diagrama de Classes	9
5.3	Diagrama de Objetos	11
5.4	Diagrama de Sequência	11
5.5	Diagrama de Atividades	12
5.6	Diagrama de Componentes	14
5.7	Diagrama de Implantação	14
5.8	Diagrama de Estados	15
6	Diagrama de Banco de Dados	16
6.1	Modelo Conceitual	17
6.2	Modelo Lógico	17
6.3	Modelo Físico	18
7	DevOps: Infraestrutura, Automação e Entrega Contínua	19
7.1	Pipeline CI/CD	19
7.2	Configuração CI/CD	20
7.3	Infraestrutura como Código	21
7.4	Infraestrutura do Sistema	21
7.4.1	Arquitetura de Rede	22
7.4.2	Diagrama de Infraestrutura	22
7.5	Arquitetura de Deploy	23
7.6	Configuração Docker Compose (Desenvolvimento)	24
7.7	Monitoramento e Alertas	24
7.8	Métricas e SLA	25
	Referências	27

Lista de Artefatos e suas Finalidades

- **CANVAS:** Modelo de negócio e estratégia
 - **SWOT:** Análise estratégica de forças, fraquezas, oportunidades e ameaças
 - **UX/UI:** Design de interface e experiência do usuário
 - **Website:** Estrutura e especificações técnicas web
 - **Diagramas UML:** Modelagem visual do sistema
 - **DevOps:** Infraestrutura, automação e práticas de entrega contínua
-

1 CANVAS: Modelo de Negócio e Estratégia

O Business Model Canvas é uma ferramenta visual estratégica que permite descrever, analisar e projetar modelos de negócio de forma clara e objetiva. O canvas é dividido em nove blocos fundamentais que cobrem as principais áreas de um negócio.

Tabela 1: Business Model Canvas - Visão Geral

Parcerias Principais	Atividades Chave	Proposta de Valor	Relacionamento	Segmentos
Fornecedores de sensores e microcontroladores (ESP32), provedores de infraestrutura em nuvem (Vercel, Render), consultores técnicos em carcinicultura	Coleta e interpretação de dados de temperatura, pH e amônia; geração de alertas em tempo real; manutenção da plataforma e dos sensores	Redução do tempo de manejo manual (de cerca de 5h/dia para monitoramento automatizado), alertas imediatos, maior precisão no controle da qualidade da água	Suporte técnico contínuo, treinamentos de uso, canal direto via chat/WhatsApp com a empresa	Pequenos e médios produtores de camarão e cooperativas de carcinicultura
	Recursos Principais		Canais	
	Sensores (DS18B20, Ph4502, MQ-135), microcontrolador ESP32, plataforma web/PWA (React/Next.js, Node/Express, MongoDB Atlas), dispensador automatizado de ração		Indicação por consultores e associações do setor, feiras e eventos de aquicultura, redes sociais e e-mail para captação, telefone/chat para suporte pós-venda	
Estrutura de Custos		Fontes de Receita		
Fixos: desenvolvimento e manutenção do sistema, hospedagem em nuvem (Vercel/Render). Variáveis: equipamentos IoT por instalação, marketing e aquisição de clientes		Assinatura mensal ou anual (SaaS), taxa de instalação do sistema físico, treinamentos avulsos		

1.1 Análise do Modelo de Negócio

A proposta de valor se apoia em resultados mensuráveis do próprio protótipo, como tempo de resposta a alertas imediata e precisão validada dos sensores, o que reduz o risco de a proposta soar genérica frente à concorrência. Ainda assim, o modelo depende de uma premissa não testada: que produtores tradicionalmente pouco digitalizados aceitem pagar recorrentemente (assinatura) por um serviço cuja alternativa (manejo manual) tem custo percebido como zero, já que é mão de obra própria. Essa é a principal tensão do Canvas e deveria ser endereçada com mais clareza, por exemplo, comparando o custo da mão de obra desviada para outras tarefas com o valor da assinatura.

A separação entre parcerias e segmentos evita um erro comum: tratar o próprio cliente (produtor de camarão) como parceiro estratégico. Os parceiros reais são quem viabiliza a operação

sem ser quem paga: fornecedores de hardware e provedores de nuvem, dos quais o negócio depende diretamente para funcionar, e consultores técnicos, que também aparecem como canal de aquisição por indicação, conectando os blocos de parceria e canal de forma coerente.

A estrutura de custos evidencia um ponto de atenção para a sustentabilidade do modelo de assinatura: os custos variáveis por instalação (sensores, microcontrolador) são custos únicos, mas o modelo de receita é recorrente, ou seja, a margem depende de reter o cliente por tempo suficiente para diluir o custo do hardware inicial, o que reforça a importância do bloco de relacionamento (suporte e treinamento) como mecanismo de retenção, não apenas de satisfação.

2 SWOT: Análise Estratégica

A Análise SWOT (Strengths, Weaknesses, Opportunities, Threats) ou FOFA (Forças, Oportunidades, Fraquezas e Ameaças) (CHIAVENATO; SAPIRO, 2020) é uma ferramenta fundamental para planejamento estratégico que permite identificar fatores internos e externos que impactam o projeto. Analisa forças, fraquezas, oportunidades e ameaças do projeto.

Tabela 2: Análise SWOT do Projeto

FATORES INTERNOS	
FORÇAS (Strengths)	FRAQUEZAS (Weaknesses)
• Tempo de resposta imediato a alertas validado em campo • Arquitetura de baixo custo (ESP32 + sensores comerciais) frente a soluções industriais de monitoramento aquícola, mais caras • Armazenamento local (NVS) no ESP32, permitindo operação temporária sem conexão contínua • Precisão de leitura validada nos sensores de temperatura, pH e amônia	• Risco de falhas técnicas no hardware (sensores, dispensador) e na integração com a plataforma • Nenhuma validação ainda com cliente pagante real, testado apenas em ambiente controlado/piloto acadêmico • Dependência de componentes importados (ESP32, sensores), sujeitos a variação cambial e prazo de importação • Curva de aprendizado para produtores pouco familiarizados com tecnologia digital
FATORES EXTERNOS	
OPORTUNIDADES (Opportunities)	AMEAÇAS (Threats)
• Crescimento do mercado de aquicultura e da demanda por rastreabilidade/sustentabilidade na produção • Parcerias com instituições de pesquisa (ex: universidades) para validação científica e captação de fomento • Expansão do sistema para outros tipos de monitoramento ambiental agrícola • Linhas de crédito e programas de incentivo à automação rural	• Cobertura <i>Wi-Fi</i> limitada em propriedades rurais do Vale do Ribeira, exigindo soluções de conectividade alternativas • Surtos de doenças no cultivo (ex: mancha branca), que afetam a demanda por qualquer tecnologia de manejo, independente de sua eficácia • Volatilidade do preço do camarão no mercado, reduzindo a margem do produtor para investir em novas tecnologias • Entrada de concorrentes com soluções similares e maior capital para escala comercial

2.1 Recomendações Estratégicas

Com base na análise SWOT, as principais estratégias devem focar em:

- **Forças + Oportunidades:** Utilizar os resultados validados (tempo de resposta, precisão dos sensores) como diferencial competitivo em parcerias com instituições de pesquisa, fortalecendo a credibilidade técnica do produto frente a linhas de fomento e programas de incentivo à automação rural
- **Forças + Ameaças:** Explorar o armazenamento local (NVS) do ESP32 e a arquitetura de baixo custo para mitigar a cobertura *Wi-Fi* limitada no Vale do Ribeira e reduzir a barreira

de investimento em regiões com produtores de menor capital, investigando futuramente conectividade LoRa/4G

- **Fraquezas + Oportunidades:** Buscar validação com cliente real via parceria com instituição de pesquisa ou programa de incentivo, transformando o piloto acadêmico em piloto comercial monitorado, o que também endereça a falta de dados de mercado
- **Fraquezas + Ameaças:** Implementar plano de manutenção preventiva dos sensores e redundância na integração de hardware para reduzir o risco de falhas técnicas, especialmente relevante diante da entrada de concorrentes com maior capital, falhas recorrentes seriam o motivo mais fácil de perder espaço de mercado

3 UX/UI: Design de Interface e Experiência do Usuário

UX (User Experience) e UI (User Interface) (ROGERS; SHARP; PREECE, 2013) são disciplinas complementares focadas em criar produtos digitais que sejam funcionais, acessíveis e agradáveis de usar.¹ O UX concentra-se na experiência geral do usuário, enquanto o UI cuida dos elementos visuais e interativos.

3.1 Guia de Estilos

Tabela 3: Paleta de Cores do Projeto

Elemento	Cor	Código HEX	Uso
Primária		#aed5ff → #ffc2c2	Botões principais, destaques
Secundária		#2261A6	Elementos secundários
Fundo		#FFFFFF	<i>Background</i> padrão
Fundo Informações		#F8fafc	<i>Background</i> de informações
Texto		#000000	Texto principal
Sucesso		#166534	Mensagens de sucesso
Erro		#991b1b	Mensagens de erro

3.2 Tipografia

Tabela 4: Tipografia do Projeto

Elemento	Fonte	Tamanho	Uso
Títulos	Montserrat	28px	Cabeçalhos de seções e páginas
Corpo	Roboto	20px	Texto principal e descrições
Botões	Poppins	16px	Botões de ação e navegação

4 Website: Estrutura e Especificações Técnicas Web

Esta seção documenta o website institucional/portfólio da equipe que desenvolveu a aplicação. O site da equipe serve para apresentar os membros, projetos realizados, competências e informações de contato.

¹Ferramentas populares para design UX/UI incluem Figma, Adobe XD, Sketch e InVision.

4.1 Mapa do Site da Equipe

O mapa do site (*sitemap*) mostra a hierarquia e organização das páginas do website da equipe:

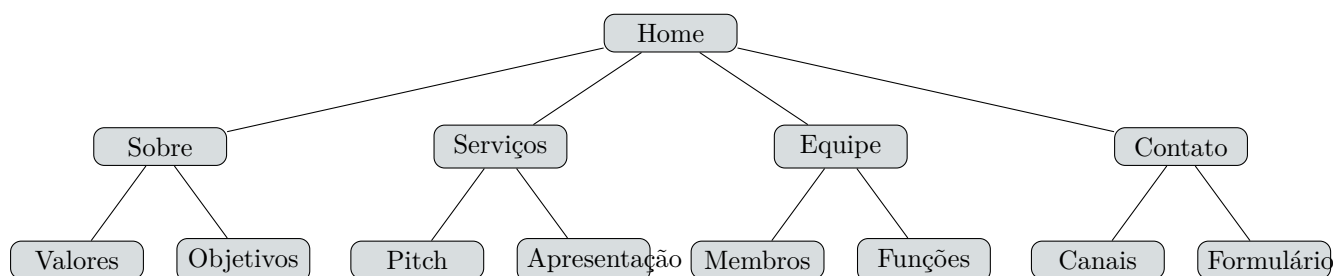


Figura 1: Estrutura de navegação do website da equipe

4.2 Conteúdo das Páginas

Páginas principais do site da equipe:

- **Home:** Apresentação inicial da equipe
- **Sobre:** Conteúdo sobre nossos valores, missão e visão
- **Serviços:** Descrição dos serviços oferecidos pela equipe
- **Equipe:** Perfil dos membros (nome, foto, função, LinkedIn/GitHub)
- **Contato:** Formulário de contato e links para redes sociais

4.3 Especificações Técnicas do Site da Equipe

Tabela 5: Stack Tecnológica do Website da Equipe

Camada	Tecnologia	Descrição
Frontend	React 18	Biblioteca JavaScript para interfaces
Build Tool e Bundler	Vite	Ferramenta de build de altíssima performance e dev server
Linguagem	TypeScript	Superset do JavaScript que garante tipagem estática segura
Estilização e Design	Tailwind CSS v4	Framework CSS utilitário para responsividade e temas
Motor de Animações	Framer Motion	Biblioteca declarativa para animações e transições complexas
Iconografia Vetorial	Lucide / React Icons	Bibliotecas de ícones modernos, responsivos e otimizados
Serviço de Mensageria	EmailJS	Plataforma <i>backend-as-a-service</i> para envio de e-mails
Hospedagem e <i>Deploy</i>	Vercel	Plataforma de <i>Edge Network</i> e <i>hosting</i> com <i>deploy</i> contínuo

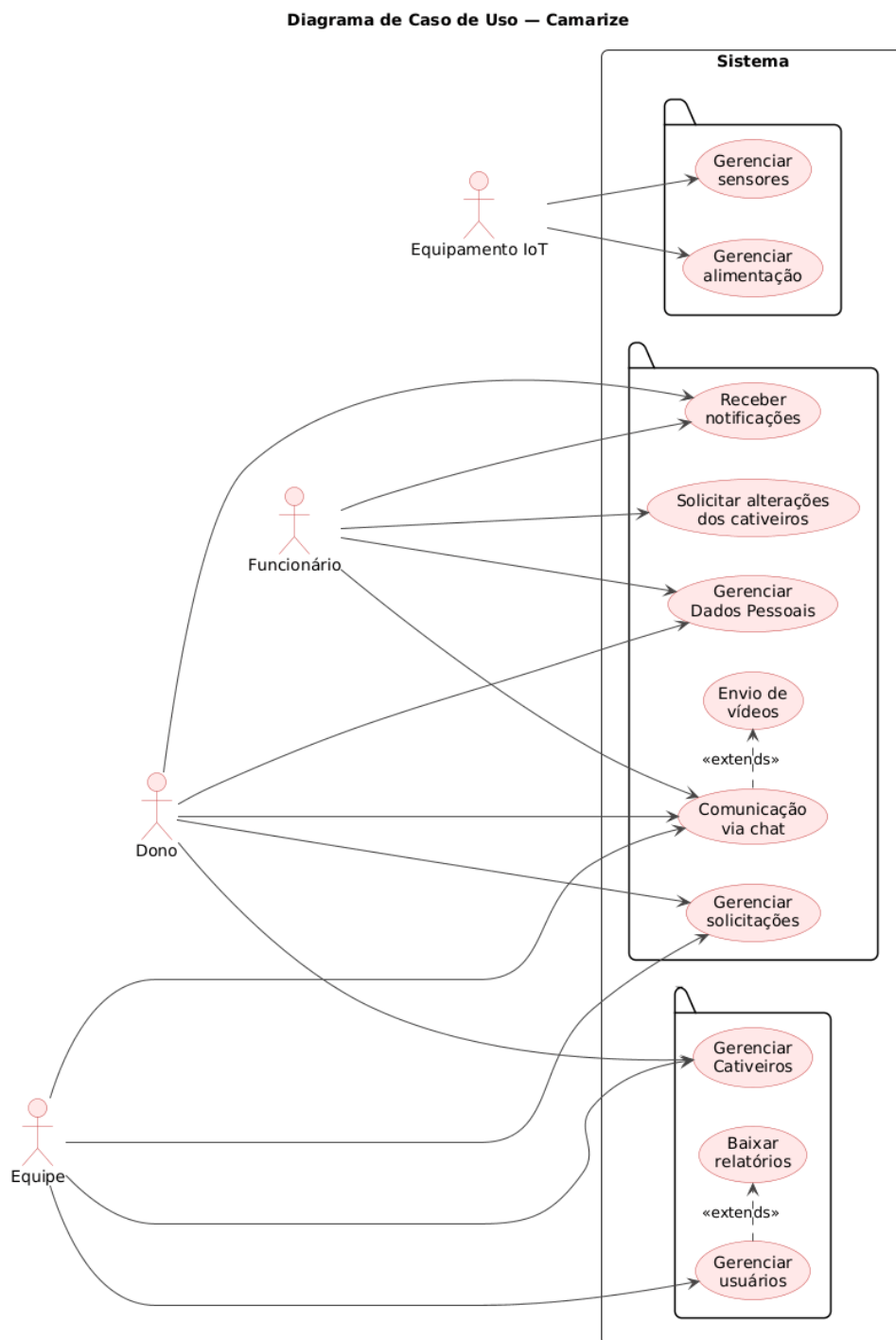
5 Diagramas UML: Modelagem Visual do Sistema

A Unified Modeling Language (UML) (BOOCH; RUMBAUGH; JACOBSON, 2005) é uma linguagem de modelagem visual usada para especificar, visualizar, construir e documentar artefatos de sistemas de *software*.

5.1 Diagrama de Casos de Uso

O diagrama identifica quatro atores principais e suas interações com o sistema: o Equipamento IoT, o Funcionário, o Dono e a Equipe técnica, cada um com permissões e funcionalidades específicas.

Figura 2: Diagrama de Casos de Uso

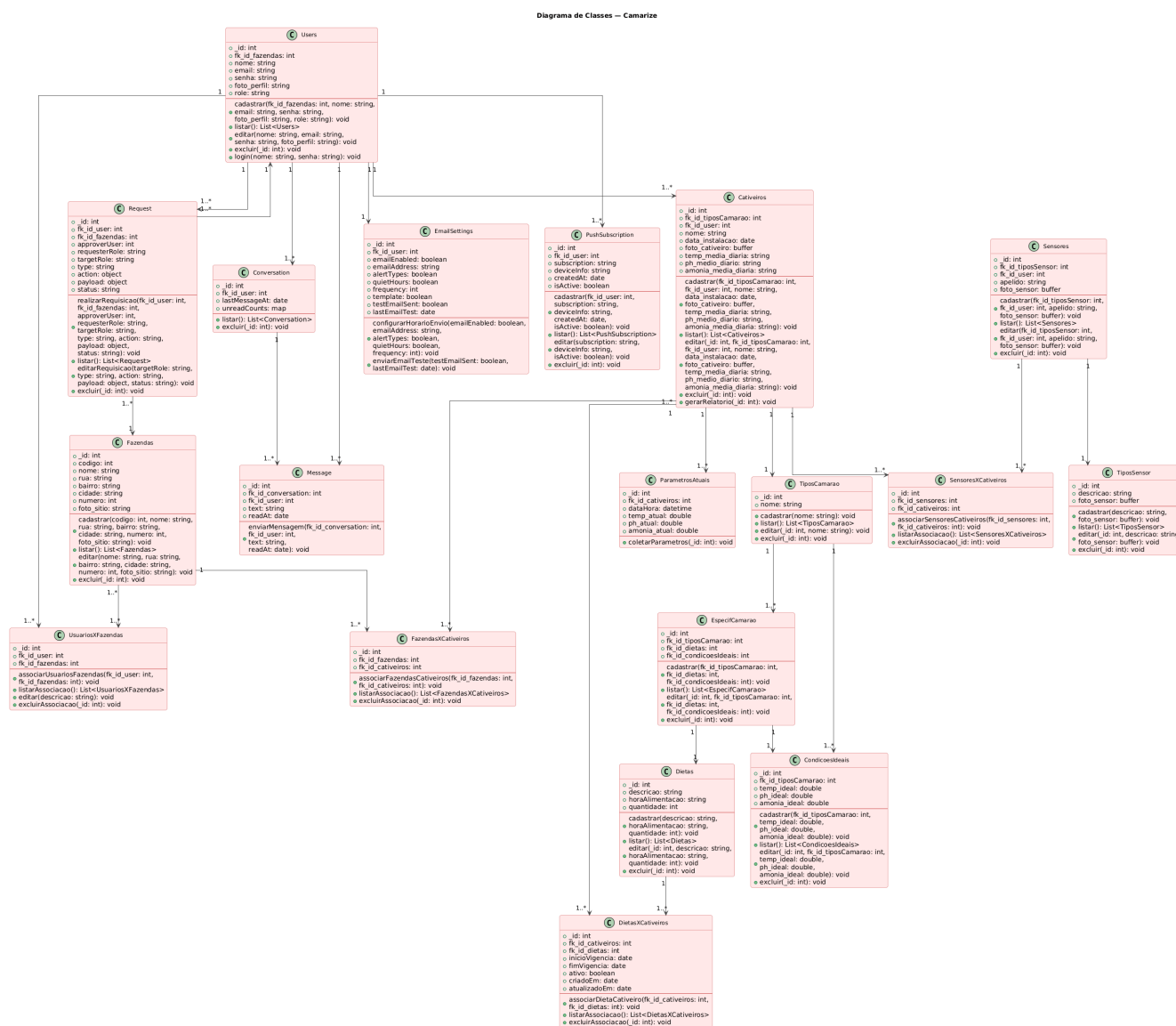


Fonte: Autoria própria

5.2 Diagrama de Classes

Apresenta a estrutura orientada a objetos do sistema, com Users como classe central e seus relacionamentos com Cativeiros, Fazendas e os módulos de comunicação e notificações. Inclui os métodos de cada classe e as associações N:N por meio de coleções intermediárias.

Figura 3: Diagrama de Classes

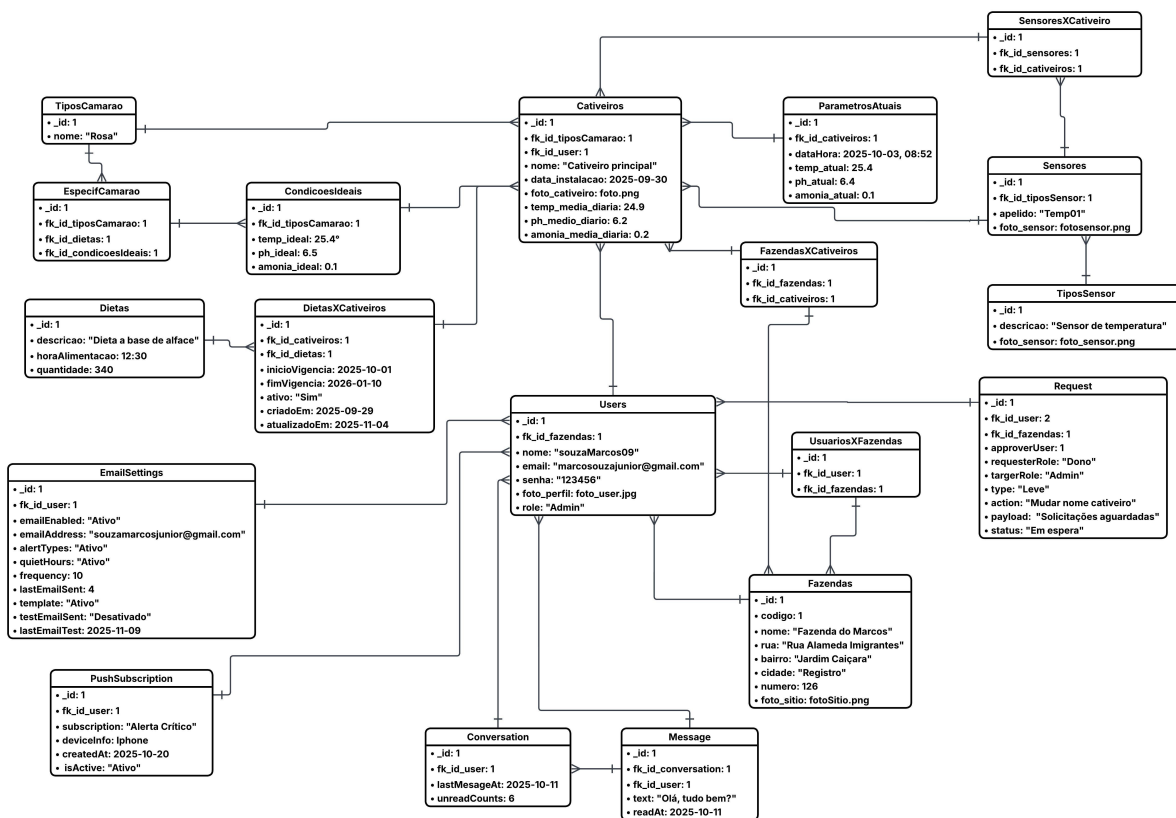


Fonte: Autoria própria

5.3 Diagrama de Objetos

O diagrama de objetos ilustra instâncias concretas das classes do sistema em um cenário de uso, exemplificando os valores dos atributos e os vínculos entre objetos em tempo de execução.

Figura 4: Diagrama de Objetos

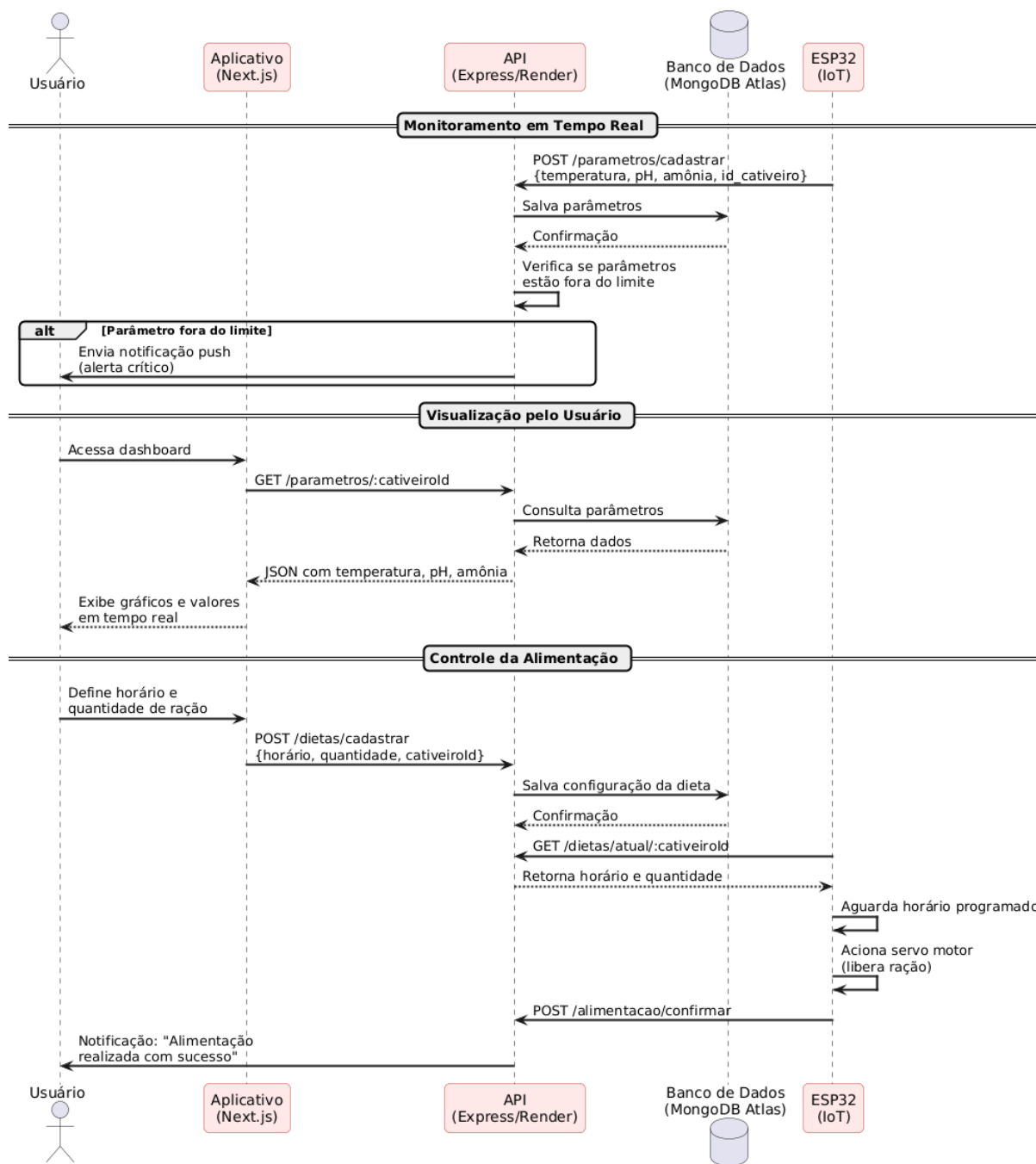


Fonte: Autoria própria

5.4 Diagrama de Sequência

Detalha três fluxos principais de interação: monitoramento em tempo real, visualização pelo usuário e controle de alimentação, envolvendo o Aplicativo, a API, o Banco de Dados e o ESP32.

Figura 5: Diagrama de Sequência

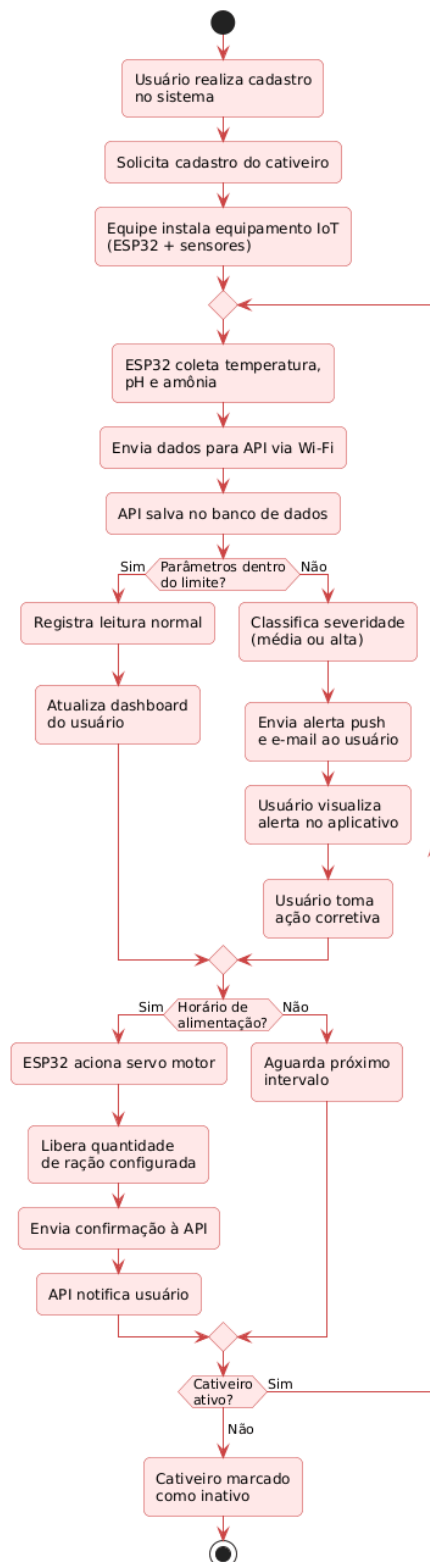


Fonte: Autoria própria

5.5 Diagrama de Atividades

Descreve o fluxo completo de operação do sistema, desde o cadastro do usuário até o ciclo contínuo de monitoramento e controle de alimentação, incluindo a lógica de verificação de limites e envio de alertas.

Figura 6: Diagrama de Atividades

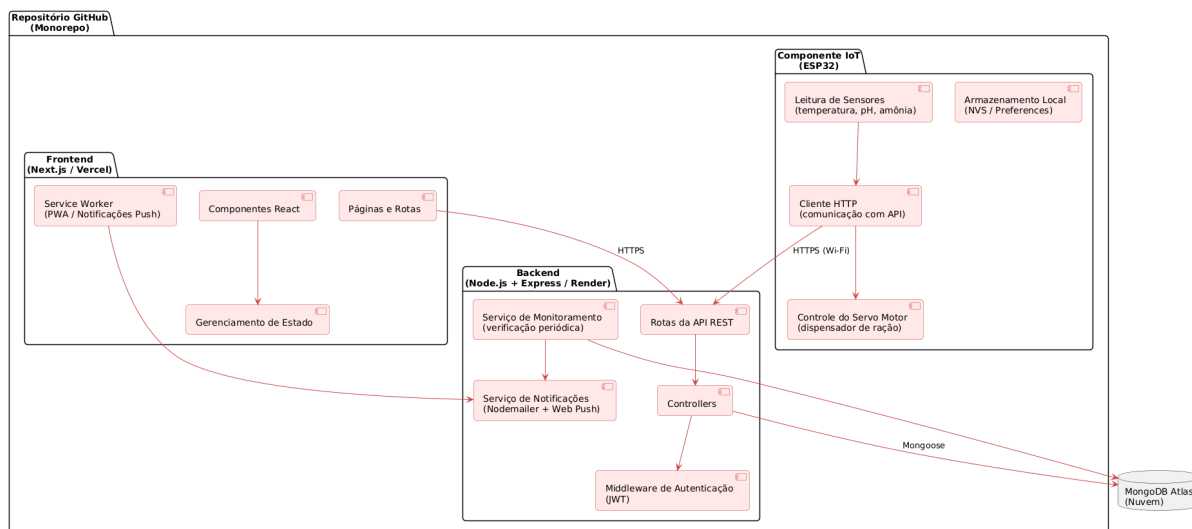


Fonte: Autoria própria

5.6 Diagrama de Componentes

Organiza o sistema em três blocos principais: Frontend (Next.js/Vercel), Backend (Node.js + Express/Render) e Componente IoT (ESP32), com comunicação via HTTPS e conexão ao MongoDB Atlas via Mongoose.

Figura 7: Diagrama de Componentes

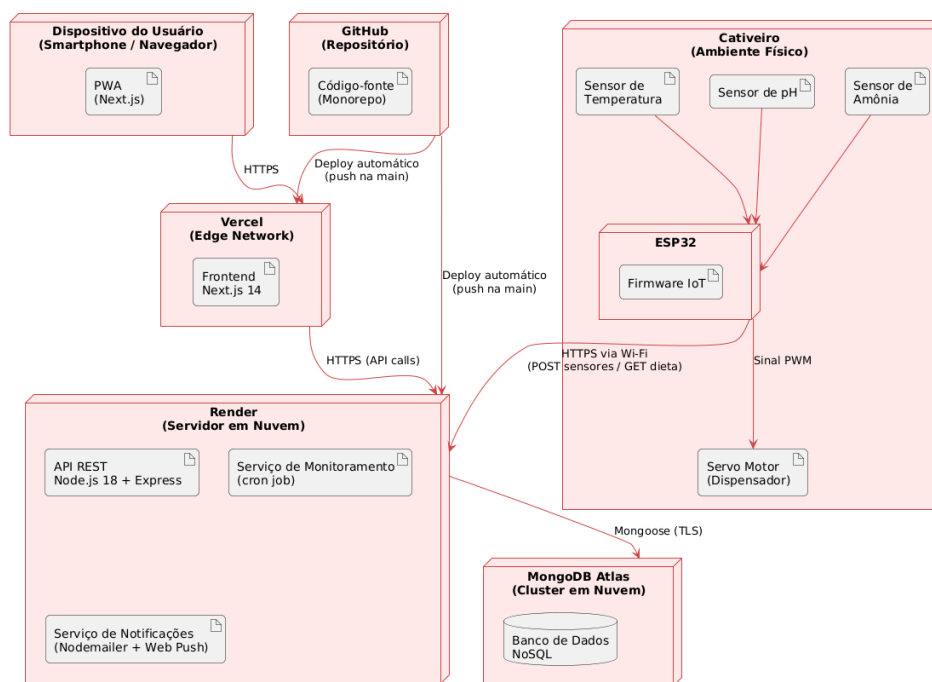


Fonte: Autoria própria

5.7 Diagrama de Implantação

Apresenta a distribuição física dos componentes em produção, incluindo a aplicação PWA na Vercel, a API REST no Render, o banco MongoDB Atlas e o ESP32 no ambiente do cativairo, com *deploy* contínuo via GitHub.

Figura 8: Diagrama de Implantação

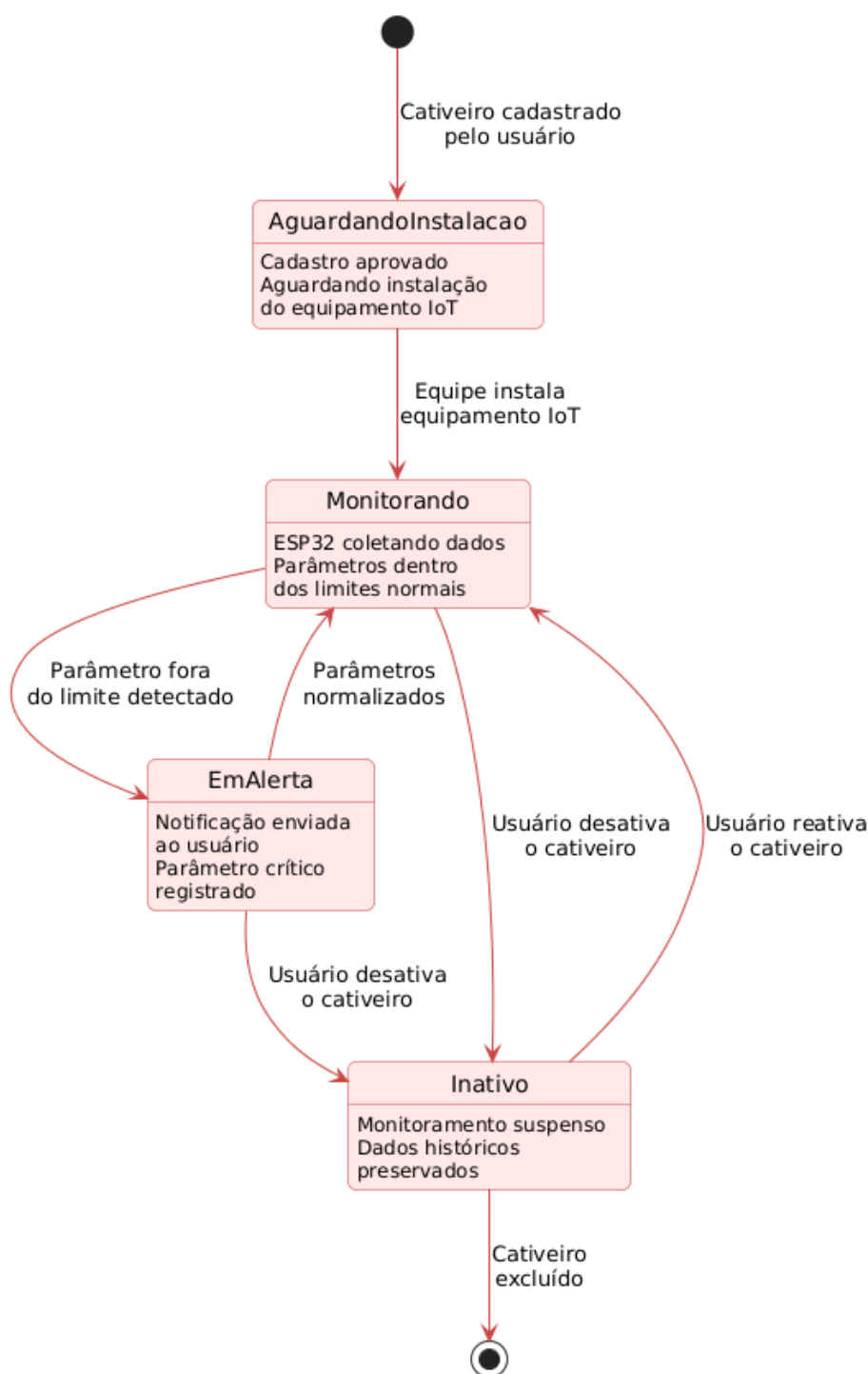


Fonte: Autoria própria

5.8 Diagrama de Estados

Modela o ciclo de vida de um cativeiro em quatro estados: AguardandoInstalação, Monitorando, EmAlerta e Inativo, com transições acionadas por eventos do sistema e ações do usuário.

Figura 9: Diagrama de Estados



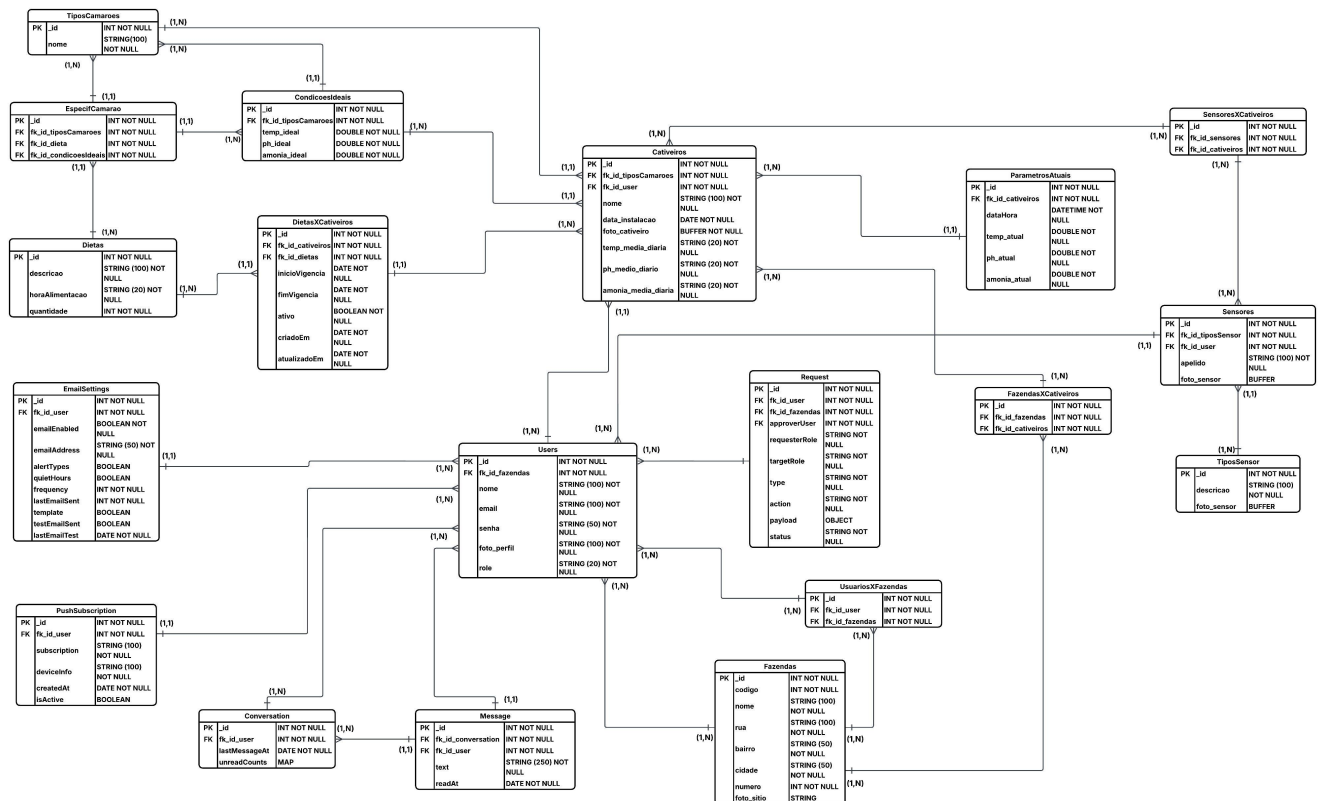
Fonte: Autoria própria

Para realizar a montagem dos diagramas, foi utilizada a ferramenta *online* PlantUML, que permite a criação de diagramas UML através de código.

6 Diagrama de Banco de Dados

O diagrama de banco de dados representa a estrutura do banco de dados não relacional (NoSQL) utilizado no projeto, mostrando as coleções, seus campos e os relacionamentos entre elas.

Figura 11: Modelo Lógico do Banco de Dados

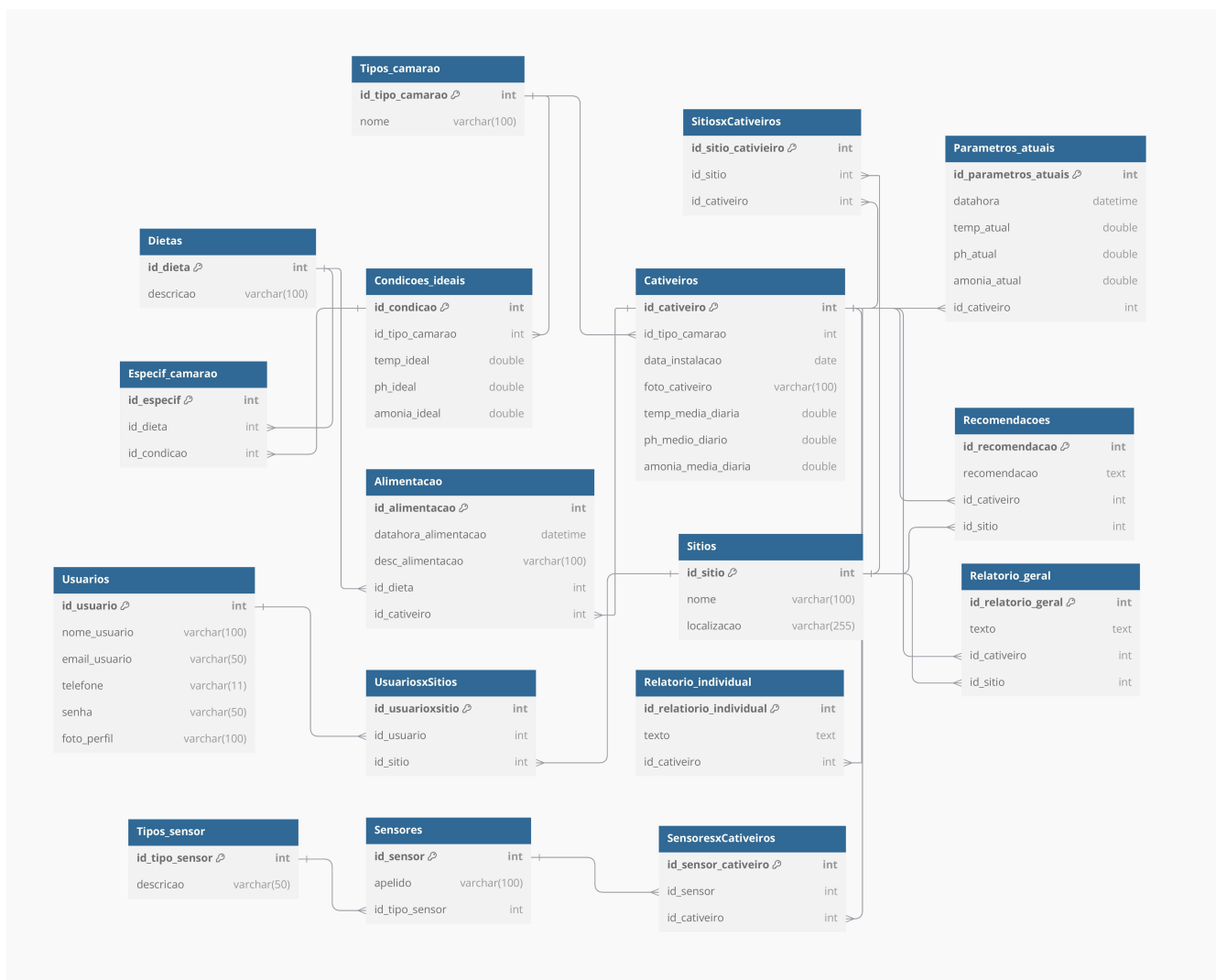


Fonte: Autoria própria

6.3 Modelo Físico

O modelo físico é a implementação final utilizando *Mongoose Schemas*, com definições de tipos (String, Number, Boolean, Buffer, Map) e restrições (`required: true`), voltado para a criação direta das coleções no MongoDB Atlas, contemplando todas as entidades do sistema incluindo EmailSettings, PushSubscription, Conversation, Message e Request.

Figura 12: Modelo Físico do Banco de Dados



Fonte: Autoria própria

7 DevOps: Infraestrutura, Automação e Entrega Contínua

DevOps (KIM et al., 2018) é uma cultura e conjunto de práticas que integra desenvolvimento de software (Dev) e operações de TI (Ops), focando em automação, colaboração e entrega contínua.²

7.1 Pipeline CI/CD

O pipeline de integração contínua é executado automaticamente a cada push na branch **main**, garantindo a qualidade do código:

²O termo "DevOps" foi popularizado por Patrick Debois em 2009.

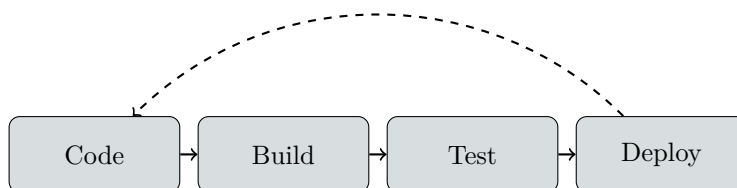


Figura 13: Pipeline CI/CD

7.2 Configuração CI/CD

O pipeline é composto por dois jobs paralelos: build do frontend (Next.js) e verificação de sintaxe da API (Node.js). Além disso, a Vercel realiza deploy automático do frontend a cada push:

```

name: CI
on:
  push:
    branches: [main]
jobs:
  build-frontend:
    name: Frontend build
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: front-react
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 18
          cache: npm
          cache-dependency-path: front-react/package-lock.json
      - run: npm ci
      - run: npm run build
  check-api:
    name: API syntax check
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 18
      - run: find api -name "*.js" -not -path "*/node_modules/*"
        -exec node --check {} \;
```

7.3 Infraestrutura como Código

Tabela 6: Ferramentas DevOps Utilizadas

Categoria	Ferramenta	Finalidade
Controle de Versão	Git + GitHub	Versionamento de código e colaboração
CI/CD	GitHub Actions	Automação de build e verificação de sintaxe
Hosting Frontend	Vercel	Hospedagem e deploy automático do Next.js
Hosting Backend	Render	Hospedagem da API Express em produção
Containerização	Docker + Compose	Empacotamento e orquestração local de serviços
Banco de Dados	MongoDB Atlas	Banco de dados NoSQL em nuvem (cluster gerenciado)
Documentação API	Swagger UI	Documentação interativa de endpoints REST
Tunelamento	Ngrok	Exposição de serviços locais para testes externos

7.4 Infraestrutura do Sistema

Esta seção documenta como o sistema funciona em diferentes ambientes e plataformas.

7.4.1 Arquitetura de Rede

Tabela 7: Componentes de Infraestrutura

Componente	Descrição
Rede Local (LAN)	• Ambiente local para desenvolvimento e testes da aplicação • Docker Compose para integração entre API (porta 4000) e Frontend (porta 3000) • Comunicação entre serviços via <code>localhost</code> com portas dedicadas • Controle de versão integrado ao Git e GitHub • Testes de sintaxe e build executados via CI antes do merge
Cloud (Nuvem)	• Frontend hospedado na Vercel com deploy automático via integração GitHub • Backend hospedado no Render como serviço web • Banco de dados MongoDB Atlas em nuvem (cluster gerenciado) • Deploy automático do frontend a cada push na branch <code>main</code> • Replicação e backup automático fornecidos pelo MongoDB Atlas • Variáveis de ambiente gerenciadas nas plataformas Vercel e Render
Dispositivos Móveis	• Interface responsiva (PWA) adaptada para smartphones e tablets • Service Worker para cache offline e notificações push • Compatibilidade com navegadores móveis modernos (Chrome, Safari) • Push Notifications via Web Push API (VAPID) • Suporte para iOS e Android via navegador
Segurança	• SSL/TLS fornecido automaticamente pela Vercel e pelo Render • Autenticação via JSON Web Tokens (JWT) com validação de <code>tokenVersion</code> • Senhas criptografadas com <code>bcrypt</code> (migração transparente de senhas legadas) • CORS configurado para origens permitidas • Controle de acesso por roles (<code>master</code>, <code>admin</code>, <code>membro</code>) • Middleware de bloqueio de escrita para membros (<code>BlockMembersWrite</code>)

7.4.2 Diagrama de Infraestrutura

Para ilustrar como os componentes se conectam:

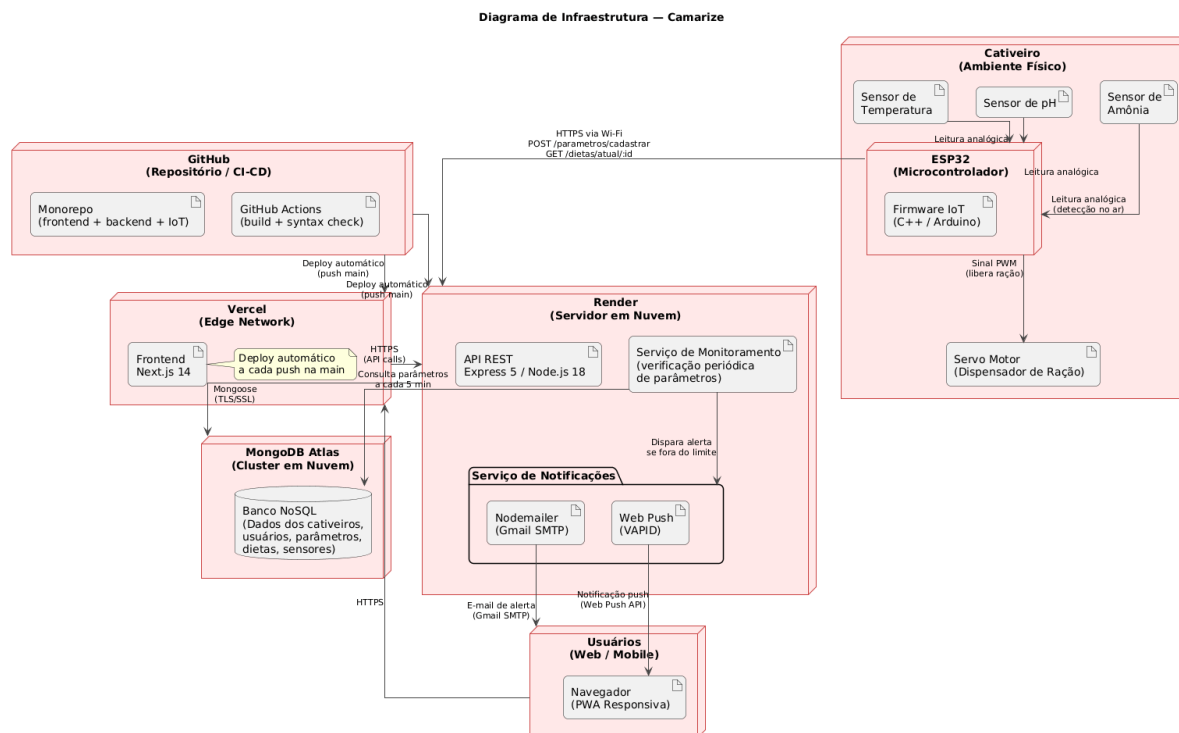


Figura 14: Diagrama de Infraestrutura do Sistema

- **Usuários Web/Mobile:** Acessam via navegador através de HTTPS (PWA responsiva)
- **Frontend:** Next.js 14 hospedado na Vercel (deploy automático)
- **Backend:** API Express 5 hospedada no Render (Node.js 18)
- **Banco de Dados:** MongoDB Atlas (cluster em nuvem)
- **Monitoramento:** Serviço interno com verificação periódica de parâmetros dos cativeiros
- **Notificações:** Nodemailer (Gmail SMTP) para alertas por e-mail + Web Push para notificações no navegador
- **ESP32:** Sensores IoT enviando dados de temperatura, pH e amônia via API REST

7.5 Arquitetura de Deploy

A arquitetura de deployment em produção utiliza serviços gerenciados na nuvem:

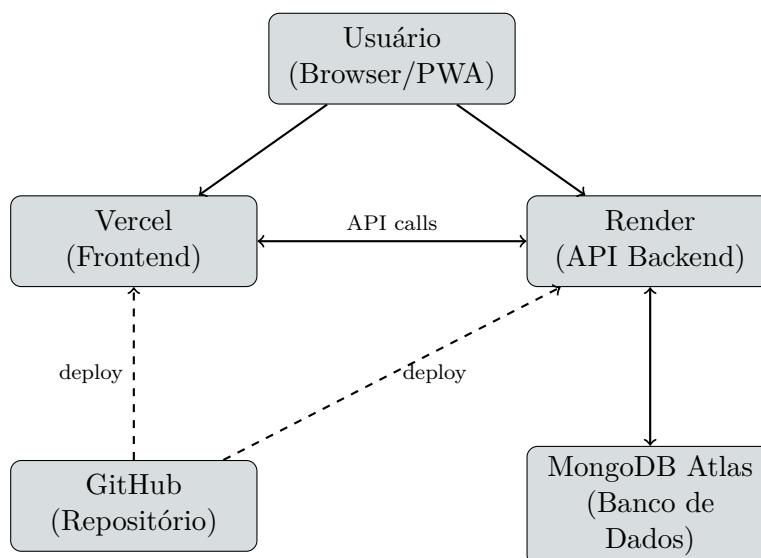


Figura 15: Arquitetura de Deployment em Produção

7.6 Configuração Docker Compose (Desenvolvimento)

Para o ambiente de desenvolvimento local, utiliza-se Docker Compose para orquestrar os serviços. Em produção, o sistema é hospedado diretamente nas plataformas Vercel (*frontend*) e Render (API), sem utilização de containers:

```

services:
  api:
    build: ../api
    container_name: camarize-api
    restart: unless-stopped
    env_file: ../api/.env
    environment:
      - NODE_ENV=development
      - PORT=4000
    ports:
      - "4000:4000"
  frontend:
    build: ../front-react
    container_name: camarize-frontend
    restart: unless-stopped
    environment:
      - NEXT_PUBLIC_API_URL=http://localhost:4000
      - NEXT_PUBLIC_SSE_URL=http://localhost:4000
    depends_on:
      - api
    ports:
      - "3000:3000"
  
```

7.7 Monitoramento e Alertas

O sistema possui um serviço de monitoramento automático que verifica periodicamente os parâmetros dos cativéis:

- **Intervalo de Verificação:** Configurável via variável de ambiente (`MONITORING_INTERVAL_MINUTES`, padrão: 5 minutos)
- **Parâmetros Monitorados:** Temperatura, pH e Amônia
- **Tolerâncias:** Temperatura (15%), pH (20%), Amônia (25%)
- **Severidade:** Classificação automática em `media` ou `alta`
- **Canais de Alerta:** E-mail (Gmail SMTP via Nodemailer) com respeito a quiet hours e rate limiting
- **Health Check:** Endpoint `/api/health` para verificação do status da API e banco de dados

7.8 Métricas e SLA

As métricas a seguir são **valores de referência** publicados pelas plataformas de hospedagem utilizadas (Vercel e Render), e não medições obtidas diretamente do projeto. Servem como parâmetros de expectativa para o comportamento da infraestrutura em produção:

- **Disponibilidade:** 99.9% (SLA garantido pela Vercel e Render em seus respectivos planos)
- **Tempo de Resposta:** < 200ms (média estimada, favorecida pela edge network da Vercel)
- **Taxa de Erro:** < 0.1% (referência da plataforma)
- **Throughput:** 1000 requisições/segundo (capacidade nominal da infraestrutura)
- **Deploy Frequency:** A cada push na branch `main` (deploy contínuo)
- **Lead Time:** < 1 hora (estimativa baseada no fluxo atual de desenvolvimento)
- **MTTR:** < 30 minutos (Mean Time To Recovery, estimativa operacional)

Conclusão

O conjunto de artefatos apresentado neste documento reflete o planejamento e a fundamentação técnica do projeto de monitoramento IoT para carcinicultura. A análise do Business Model Canvas, complementada pela interpretação estratégica dos nove blocos, evidenciou um modelo de negócio centrado na automação e na redução de custos operacionais para produtores de camarão. A análise SWOT identificou a dependência de conectividade e o custo inicial como principais desafios internos, contrabalançados pelo crescimento do mercado de aquicultura e pela demanda por soluções sustentáveis.

A seção de UX/UI foi estruturada com guia de estilos, tipografia, componentes visuais, persona do usuário-alvo e jornada de uso, fundamentando as decisões de *design* na pesquisa de campo realizada com produtores do Vale do Ribeira.

No âmbito técnico, os sete diagramas UML documentam a arquitetura completa do sistema, desde os casos de uso até a infraestrutura de implantação, fornecendo uma visão abrangente das interações entre os componentes. A modelagem do banco de dados não relacional (MongoDB) foi detalhada nos níveis conceitual, lógico e físico, refletindo a implementação real do sistema com *Mongoose Schemas*. A infraestrutura DevOps implementada, com *pipeline* CI/CD via GitHub Actions, containerização com Docker para desenvolvimento local e *deploy* contínuo nas plataformas Vercel e Render, garante um fluxo de entrega automatizado e rastreável.

O website institucional da equipe complementa a documentação ao padronizar a identidade visual e os canais de comunicação do projeto. Em conjunto, os artefatos formam uma base documental coerente que sustenta o desenvolvimento, a manutenção e a evolução futura do sistema.

Referências

BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. *UML: Guia do Usuário*. 2. ed. Rio de Janeiro: Elsevier, 2005.

CHIAVENATO, I.; SAPIRO, A. *Planejamento Estratégico: da Intenção aos Resultados*. 4. ed. São Paulo: Atlas, 2020.

KIM, G. et al. *Manual de DevOps: Como Obter Agilidade, Confiabilidade e Segurança em Organizações Tecnológicas*. Rio de Janeiro: Alta Books, 2018.

ROGERS, Y.; SHARP, H.; PREECE, J. *Design de Interação: Além da Interação Humano-Computador*. 3. ed. Porto Alegre: Bookman, 2013.